



Manual de uso

Versão 3.25.0 · setembro de 2026
WINDEV · WEBDEV · WINDEV Mobile

Manual de uso — WX Claude Code

Plugin do Claude Code para converter um projeto **WINDEV**, **WEBDEV** ou **WINDEV Mobile** para outra linguagem sem inventar o que o projeto faz. Oito capítulos, na ordem em que você vai precisar deles.

1. Como instalar
2. Comandos  do plugin
3. Como funciona a gerência de projeto
4. Como funciona a economia de tokens
5. Como subir os arquivos
6. Como invocar o wizard
7. Como definir a linguagem e a plataforma de destino
8. Licença e serial de ativação

1. Como instalar

Requisitos. Claude Code; Python 3.10 ou mais novo; Node 22 ou mais novo (só para o Impeccable, o módulo de qualidade gráfica). Para extrair texto de PDF, `pip install pypdf`.

Pelo marketplace (recomendado):

```
claude plugin marketplace add adrianoboller/adrianoboller
claude plugin install wx-claude-code@wx-claude-code
```

Pelo zip. O plugin vem em dois arquivos, porque o completo passa de 30 MB: `wx-claude-code-<versão>-plugin-sem-corpus.zip` e `Help_WL_12k_Json-corpus-do-plugin.zip`.

1. Descompacte o primeiro numa pasta, por exemplo `~/plugins/`. Ele cria `wx-claude-code/` e `.claude-plugin/`.
2. Copie o `Help_WL_12k_Json.zip` do segundo para `wx-claude-code/skills/conversao-wx/resources/`. Não descompacte.
3. Confira o corpus:

```
python3 wx-claude-code/skills/conversao-wx/scripts/query_wlanguage_help.py --verify
```

O hash tem de ser `a95ed553...` e o estado `DEGRADED/CONDITIONAL` (três defeitos conhecidos e documentados; não é erro de instalação).

1. Carregue sem instalar, para testar: `claude --plugin-dir ~/plugins/wx-claude-code`. Ou instale de vez: `claude plugin marketplace add ~/plugins` e `claude plugin install wx-claude-code@wx-claude-code`.

Conferir. Numa sessão nova, peça «liste as skills e os agentes com prefixo `wx-claude-code:` ». Devem aparecer 5 comandos, 3 skills e 94 agentes. A listagem que o modelo devolve pode omitir um item; confira por nome, não por contagem.

Validar o pacote (roda os 27 testes de regressão):

```
python3 wx-claude-code/skills/conversao-wx/scripts/validate_plugin_bundle.py wx-claude-code --strict
claude plugin validate wx-claude-code
```

2. Comandos / do plugin

COMANDO	O QUE FAZ	QUANDO
<code>/wx-claude-code:questionario <projeto></code>	o wizard: bloco 0 e letras A–L, um item por mensagem; gera <code>.wx-migration/</code> e o contexto do projeto	sempre primeiro
<code>/wx-claude-code:converter <modo> <projeto></code>	conversão por gates G0–G7; modo é <code>inventario</code> , <code>plano</code> , <code>piloto</code> ou <code>completo</code>	depois do wizard
<code>/wx-claude-code:pmo <ação> <projeto></code>	gerência: <code>iniciar</code> , <code>bloco</code> , <code>sprint</code> , <code>identificacao</code> , <code>status</code> , <code>relatorio</code> , <code>kanban</code> , <code>pdca</code> , <code>orcamento</code> , <code>entregar</code> , <code>painel</code> , <code>exportar</code> , <code>limpar</code>	durante toda a conversão
<code>/wx-claude-code:estilo-telas <projeto></code>	paleta, tema e tipografia viram <code>PRODUCT.md</code> e <code>DESIGN.md</code> pelo Impeccable	quando a letra F foi «sim»
<code>/wx-claude-code:laudo-tokens [fase]</code>	auditoria de consumo em três fases, somente leitura	quando quiser medir o custo
<code>/impeccable <comando> <alvo></code>	os comandos de qualidade gráfica (23 segundo o <code>SKILL.md</code> de origem): <code>shape</code> , <code>polish</code> , <code>audit</code> , <code>critique</code> , <code>harden</code> ...	em cada tela convertida

Os comandos têm scripts por trás, que você também pode rodar direto. Todos ficam em

`$CLAUDE_PLUGIN_ROOT/skills/conversao-wx/scripts/`:

SCRIPT	FAZ
<code>aplicar_questionario.py</code>	respostas do wizard viram o <code>.wx-migration/</code> , o contexto do projeto (<code>CLAUDE.md</code> , <code>INDEX_FILES.md</code> , <code>.claude/</code> , prompts) e o ambiente (letra K)
<code>wx_preflight.py</code>	Gate G0: confere cada anexo fisicamente
<code>extrair_pdf.py</code>	texto por página com <code>arquivo#page=N</code> e hash
<code>query_wlanguage_help.py</code>	busca no corpus do Help por símbolo e tema
<code>golden.py</code>	captura resultados do legado e compara com o novo
<code>rotear_modelo.py</code>	escolhe o modelo Claude por classe de tarefa e orçamento
<code>pmo.py</code>	plano, sprint, kanban, PDCA, painel, entrega
<code>uso_de_tokens.py</code>	lê o consumo real das sessões e lança no orçamento
<code>verificar_ambiente.py</code>	mede o que está instalado contra o mínimo pedido em K
<code>licenca.py</code>	serial de ativação: chaves, gerar, instalar, verificar, hooks
<code>safe_unpack_bundle.py</code>	descompacta anexo zipado com defesa contra travessia e zip bomb
<code>exportar_projeto.py</code>	salva o projeto resultante organizado, com manifesto de hashes, na pasta do usuário
<code>zelador.py</code>	limpa temporários uma vez por dia; nunca toca anexo, matriz, PMO ou código

Atalhos. Crie os seus em `.claude/commands/<nome>.md` no projeto. Exemplo para polir telas com as regras da conversão já embutidas:

```
---
description: Polir uma tela convertida preservando campos, textos e paleta
argument-hint: "<caminho-da-tela>"
---
Carregue a skill impeccable e execute `polish $ARGUMENTS`.
Preserve a ordem dos campos, os textos e as validações do legado.
Cores só do DESIGN.md; contraste mínimo 4,5:1, e diga o valor medido.
```

Aí `/polir-tela src/telas/Venda.tsx` faz a passada. O Impeccable também cria atalhos sozinho: `node "$CLAUDE_PLUGIN_ROOT/skills/impeccable/scripts/pin.mjs"` `pin audit` gera `/audit`.

3. Como funciona a gerência de projeto

O PMO é código, não texto. Quem responde «em que pé está, quanto custou, o que trava, quem decide» é o agente `pmo-gerente-de-projetos` rodando `pmo.py`. Nenhum número do painel é digitado: cada linha cita a fonte, e o que não tem fonte aparece como `INDISPONÍVEL`, nunca como zero.

Começar:

```
/wx-claude-code:pmo iniciar ./meu-projeto
```

Cria `.wx-migration/pmo/` com plano por gate, orçamento, RAID (riscos, premissas, issues, dependências), backlog, base de conhecimento e a pasta de ciclos PDCA.

O relatório sai no fim, sozinho. Fechar uma sprint (`pmo.py sprint fechar`) e entregar ao stakeholder (`pmo.py entregar`) geram `pmo/relatorio.md` e `pmo/painel.html` sem ninguém pedir. O relatório tem onze seções, todas lidas dos arquivos: empresa e contrato (prazo com os dias restantes, orçamento, marcos), o painel de gates, a rastreabilidade por tipo com os itens bloqueados, as tabelas inteiras de lacunas, decisões e riscos, a história das sprints com a vazão medida, os ciclos PDCA, o roteamento por classe e modelo, a estratégia de conversão e o destino da entrega, e os próximos passos derivados do que está acima (próximo gate, sprint aberta, itens a desbloquear, decisões pendentes, lacunas críticas, prazo vencido). `pmo.py relatorio` imprime o mesmo texto a qualquer hora.

Hooks e RAG. O harness faz valer sozinho o que antes era só regra escrita: anexos são somente leitura (o hook nega escrita e `rm` na pasta de evidências), segredo nunca vai para arquivo (nega conteúdo com formato de token, gravação de `.env` e `git add .env`), o Kanban se rege quando a matriz ou o backlog mudam, e a cada pergunta sua o RAG do projeto injeta os trechos mais próximos com `arquivo#linha` para o Claude Code abrir o arquivo certo em vez de ler tudo. O RAG é `rag.py` (`indexar`, `buscar`), BM25 em Python puro sobre `.wx-migration/`, os arquivos de contexto e as referências do plugin; o corpus WLanguage continua por tema no `query_wlanguage_help.py`. Tabela completa em `references/hooks-e-rag.md`.

A equipe prioritária. Dez agentes que têm de existir num projeto de grande porte, na ordem de prioridade, cada um com o seu gatilho: A Zelador (sinal de falta de espaço ou hook diário); B Pesquisador (todo ciclo PDCA infrutífero vira um pedido em `pmo/pesquisas.md`, e ele busca na internet e responde com a fonte); C Documentador (`documentar_codigo.py` gera `funcoes.md`, `funcoes.html` e `indice.json` com

finalidade, parâmetros, processamento e resultados de cada função); D Supervisor de qualidade (confronta fonte, documentação, objetivos e as interjeições do stakeholder em `pmo/interjeicoes.md`); E Gestor de tarefas (`pesar_tarefa.py` pesa por linhas e tempo de tarefas similares e escolhe o modelo, do barato ao mais caro); F GP (backlog, Kanban, versionamento por sprint); G Equipe de testes (roda as baterias e entrega ao conferente de prova real); H Status (`pmo.py status --por-agente` , a partir do que cada agente registra com `pmo.py atividade`); I Gestor da base de conhecimento (`pmo/conhecimento/frutiferos.md` , `infrutiferos.md` , `indice.md` , com aviso ao GP); J Tradutor multilíngue, que só entra a pedido e centraliza os textos em `i18n/textos.json` . Detalhe em `references/equipe-prioritaria.md` .

Blocos, sprints e a identificação de cada interação. O projeto se divide em blocos numerados (`Bloco0001` , `Bloco0002` ...), os capítulos, e cada bloco tem sprints numeradas (`SP00001` , `SP00002` ...). Toda resposta do Claude Code no projeto começa com a linha `BlocoNNNN-SPNNNN-Título · data` , por exemplo `Bloco0001-SP00001-Análise da base de dados · 2026-09-03` : o hook a injeta a cada mensagem e os comandos exigem que ela abra a resposta. Ao fechar, cada sprint deixa `pmo/sprints/Bloco0001-SP00001-analise-da-base-de-dados.md` e a cópia zipada ao lado, com o mesmo nome.

Salvar o projeto resultante numa pasta sua. `/wx-claude-code:pmo exportar` (ou `exportar_projeto.py --destino <pasta>`) grava em `<pasta>/<nome>-<data>/` sete pastas numeradas: questionário, evidências (por hash, ou copiadas com `--com-evidencias`), inventário e decisões, PMO, ambiente e prompts, código e relatório final, com um `00-LEIA-ME.md` e um `manifesto.json` que tem o SHA-256 de cada arquivo. `.env` , chaves, `target/` , `node_modules/` e `.git/` ficam de fora; arquivo com formato de token é recusado com o caminho. A pasta pode vir do questionário (`L3.pasta_de_saida`).

O zelador limpa os temporários. Uma vez por dia, ao abrir a sessão, o hook roda `zelador.py` e apaga só o que é temporário: execuções antigas do pré-flight (ficam as três últimas), logs com mais de 7 dias, `__pycache__` e worktrees parados. Anexos, matriz, decisões, PMO, entregas e código não entram. Cada rodada fica em `.wx-migration/logs/zelador.md` com os bytes medidos; `/wx-claude-code:pmo limpar` roda na hora, e sem `--executar` só relata.

O que o PMO já recebe do wizard. O bloco 0 do questionário (capítulo 6) deixa em `pmo/` o cronograma com o prazo final, o organograma, o fluxograma, os riscos iniciais (`RSK-*`) e `projeto.json` com o orçamento financeiro. O `iniciar` lê esse arquivo e preenche `previsto_para` de cada gate que tem marco; o `status` mostra quantos dias

faltam para o prazo final e o orçamento aprovado, ou `INDISPONÍVEL` se não foram informados. O relatório também lê `processo-de-conversao.md` (estratégia), `entrega.json` (destino, sem credencial) e `ambiente.md`, todos do mesmo wizard.

Gates. O trabalho avança em oito portões, G0 a G7, e cada um depende de um aprovador humano:

GATE	O QUE ACONTECE	QUEM APROVA
G0	anexos conferidos fisicamente (pré-flight)	responsável pelos anexos
G1	inventário, hashes, extração de texto, índice do Help	líder técnico
G2	regras, telas, dados, queries, integrações, conflitos	responsável de negócio
G3	arquitetura-alvo, decisões, plano de ondas, rollback	arquitetura
G4	piloto vertical: uma fatia com tela, regra, query e erro, comparada ao legado	técnico + negócio
G5	ondas de implementação por módulo	líder técnico
G6	segurança, desempenho, concorrência, testes	qualidade
G7	ensaio, reconciliação, cutover e plano de retorno	patrocinador

O piloto (G4) nunca é pulado numa conversão completa. Quem implementa não aprova: o `quality-auditor` tenta refutar, o humano decide.

Os dez papéis. Em projeto de grande porte o trabalho é distribuído por papéis com dono: A orquestrador, B engenheiro, C DBA, D zelador, E designer, F prova real, G QA, H documentação, I versionador, J pesquisador. Cada papel tem quatro subagentes, Plan, Do, Check e Act, e executa todo item como um ciclo PDCA.

Scrum. Uma sprint por gate ou onda. O PMO abre a sprint atribuindo o papel dono de cada item do backlog:

```
pmo.py sprint abrir --nome "Onda 1 · vendas" --objetivo "..." --gate G5 --item QRY-001:C --item UI-001:E --item BR-003:F
pmo.py sprint fechar --decisao APPROVED|CONDITIONAL|REJECTED --pedido "..."
```

O fechamento escreve o resumo de doze seções em `pmo/sprints/` e devolve ao backlog o que não atingiu a definição de pronto: evidência com localizador, implementação apontada, teste, resultado comparado, aprovação humana, confiança nunca `low`.

Kanban. `pmo.py kanban` gera o quadro da matriz de rastreabilidade com o papel em cada cartão (`[C dba] QRY-001 ...`) e limite de WIP (6 em andamento, 4 em verificação).

Coluna estourada não recebe cartão; item [sem papel] ninguém pega até o PMO atribuir. O quadro não se edita: muda-se o estado na matriz e o papel no backlog.

PDCA. Toda hipótese de trabalho abre um ciclo com critério numérico e fecha como frutífero ou infrutífero, gravando uma linha na base de conhecimento nos dois casos. Infrutífero sem a próxima hipótese não fecha.

```
pmo.py pdca abrir --gate G4 --hipotese "..." --medida "..." --criterio "ganho >= 1,5x"
pmo.py pdca fechar --id PDCA-001 --resultado infrutifero --medido "1,06x" --aprendizado "..." --proxima "..."
```

Entrega ao stakeholder. Fechada a sprint:

```
pmo.py entregar --sprint 2 --plugin-root "$CLAUDE_PLUGIN_ROOT"
```

Gera pmo/entregas/sprint-02-G5-<data>.zip com o resumo da sprint, as técnicas aplicadas com números e fonte, a base de conhecimento, o que cada ferramenta faz (lido do cabeçalho de cada script), decisões, lacunas, RAID, backlog e o Kanban do fechamento.

Painel. pmo.py status regenera o texto e pmo.py painel gera o HTML para o aprovador abrir no navegador, em tema claro ou escuro.

O corpus no CLAUDE.md e no RAG. O CLAUDE.md gerado tem a seção «Corpus WLanguage 12k»: o que é, o comando de consulta por tema e a regra de nunca usá-lo como regra de negócio. O RAG reconhece símbolos WLanguage citados na pergunta e injeta o tema e o comando exato, lendo só os nomes dos membros do zip (7.265 símbolos, 72 ms).

4. Como funciona a economia de tokens

Três mecanismos, todos medidos.

Balanceamento de modelos. Antes de cada delegação o orquestrador chama

`rotear_modelo.py` com a classe da tarefa e os sinais de risco. Haiku faz o mecânico (hash, contagem, busca no corpus), Sonnet analisa e implementa, Opus decide e revisa. Conflito, fiscal, permissão ou dado pessoal sobem um degrau; padrão já aprovado ou volume grande descem. Acima de 80 % do orçamento do gate rebaixa; acima de 100 % bloqueia e o PMO decide com o número.

Orçamento medido, não estimado. O Claude Code grava o consumo de cada resposta.

`uso_de_tokens.py` lê esses registros, deduplica por mensagem e lança no orçamento do gate:

```
uso_de_tokens.py --project-root . resumo
uso_de_tokens.py --project-root . lancar --gate G4
```

Numa sessão real deste projeto, um único `/wx-claude-code:pmo status` delegado a um subagente custou 305.883 tokens. É esse tipo de número que o orçamento por gate precisa ver.

Hábitos que o plugin impõe. Anexos e corpus são consultados por índice, nunca abertos inteiros. Cada especialista WLanguage lê só a sua fatia do Help (`--group`), o que reduziu uma busca de 5,4 s para 0,5 s. Saída longa de comando vai para `.wx-migration/logs/` e volta como localizador. Quando a letra J do wizard é «sim», o `CLAUDE.md` do projeto recebe o estilo de resposta direto ao ponto.

Laudos. `/wx-claude-code:laudo-tokens` audita em três fases. A primeira é somente leitura e termina numa tabela de problemas por impacto; então para e espera o seu OK. A segunda propõe uma mudança por vez. A terceira entrega até três hábitos, só os que tiverem evidência nas suas sessões. Todo número é `MEDIDO`, `ESTIMADO` ou `INDISPONÍVEL`.

5. Como subir os arquivos

O plugin não lê o projeto WINDEV binário. Ele lê o que a plataforma exporta. Crie uma pasta de evidências dentro do projeto de destino, por exemplo `inputs/`, e coloque nela:

ARQUIVO	O QUE É	COMO GERAR NO WINDEV
<code>banco.sql</code>	DDL do banco: tabelas, índices, constraints, triggers, views	análise → exportar script SQL, ou dump do HFSQL
<code>codigo.pdf</code>	só procedures, classes e eventos	documentação técnica → filtrar código
<code>interfaces.pdf</code>	só janelas, páginas, controles e relatórios	documentação técnica → filtrar telas
<code>queries.pdf</code>	só as queries: nome, SQL, parâmetros, onde são usadas	documentação técnica → filtrar queries
<code>completo.pdf</code>	a documentação técnica inteira; serve de reserva se algum dos três faltar	documentação técnica → tudo
<code>screenshots/*.png</code>	cada tela em cada estado (normal, vazio, erro)	capturas
<code>screenshots/screenshots.json</code>	para cada captura: <code>arquivo</code> , <code>tela</code> , <code>estado</code> , <code>plataforma</code>	à mão
<code>dados-de-amostra/</code>	dados sintéticos e resultados esperados do legado (golden master)	à mão ou exportação anonimizada
<code>marca/*.svg</code> ou <code>.png</code>	logotipo da empresa e do software, e organograma ou fluxograma se existirem como arquivo	do departamento de marketing

Regras:

- Os PDFs precisam ser **pesquisáveis** (texto, não imagem). Sem isso a extração exige OCR e o pré-flight marca `OCR_REQUIRED`.
- Os anexos são **somente leitura**. O plugin nunca escreve neles; tudo que gera vai para `.wx-migration/`.
- Nada de senha, token, certificado ou dado real de pessoa. Dados de amostra são sintéticos ou anonimizados. A credencial do GitHub de destino fica na máquina (variável de ambiente, `gh_auth`); o questionário guarda só o nome.

- Anexo dentro de zip: o plugin descompacta com `safe_unpack_bundle.py` numa pasta nova, com defesa contra travessia de caminho e zip bomb.

O projeto de exemplo `exemplos/estoque-wx/` tem tudo isso montado. Use-o como modelo da pasta e como ensaio antes do seu projeto.

6. Como invocar o wizard

O wizard é o questionário: o bloco 0 da empresa e do projeto, e as letras A–L. É sempre o primeiro comando de um projeto:

```
/wx-claude-code:questionario ./meu-projeto
```

Como ele se comporta. Pergunta **uma letra por mensagem** e espera. Você responde, ele confirma em uma linha o que registrou (A: inputs/banco.sql, HFSQL 2025 → provided) e só então faz a próxima. A resposta decide o caminho: sem o PDF de código em B, ele avisa em E que o completo vai cobrir; «não» ao Impeccable em F pula paleta e tipografia; mobile em I pede versões de Android e iOS. Quem não tem um item responde «não tenho», e isso vira `missing` no manifesto, nunca `not_applicable` por inferência.

Um caminho só conta como fornecido depois que o wizard **abre o arquivo**.

Bloco 0, antes da letra A. Dezesseis perguntas sobre quem pede e o que é o projeto, uma por mensagem:

ITEM	PERGUNTA
0.1	softhouse solicitante (razão social, fantasia, CNPJ) e a solicitação em uma ou duas frases
0.2	diretores: nome, cargo, contato
0.3	endereço completo
0.4	logotipo da empresa (arquivo na pasta de evidências)
0.5	logotipo do software
0.6	finalidade do software
0.7	objetivos do projeto
0.8	descrição do software, recursos e módulos
0.9	organograma: arquivo ou as posições (papel, nome, responde a)
0.10	fluxograma: arquivo ou as etapas em ordem (vira Mermaid)
0.11	cronograma: início, marcos com data e gate, prazo final de entrega
0.12	orçamento: valor, moeda, base e quem aprovou
0.13	riscos conhecidos: probabilidade, impacto, resposta, dono
0.14	peçoal envolvido
0.15	GitHub de destino: URL, branch, usuário, nome da credencial e diretório de destino
0.16	quem aprova: nome, cargo, e-mail, o que aprova e o substituto

A letra K, o ambiente. Para cada ferramenta marcada o script gera `.wx-migration/ambiente/instalar-ambiente.sh` (rustup para o Rust, o pacote do banco, `git` e `gh` para o GitHub, tudo idempotente), o SQL dos papéis do banco por nível (`superuser`, `owner`, `readwrite`, `readonly`) e um `.env.exemplo` sem valores. O login do root e de cada papel é perguntado; a senha, não: você diz em que variável de ambiente ela vai ficar, e o SQL usa `${VARIABLE}`. O `verificar_ambiente.py` mede o que já está instalado contra a versão mínima e devolve 3 quando falta algo.

Root e sudo. O instalador confere se já é root; se não, usa `sudo` quando existe (e pede a senha uma vez com `sudo -v`), ou entra como root com `su` quando você disse em K0 que não há sudo. Só os passos que exigem root passam por aí: instalar pacote e habilitar serviço. Rustup, git e o SQL dos papéis rodam como você.

n8n. Marcado em K7, o script gera `ambiente/n8n/docker-compose.yml` (ou a instalação por npm), o SQL do banco do n8n no PostgreSQL de K2 e o `integracao.md`, que lista os webhooks que o n8n expõe, os fluxos iniciais e o que o projeto precisa ter

para conversar com ele: cliente HTTP com retry para cada evento, uma rota de API por ação, e um `INT-*` na rastreabilidade para cada um. Chave de criptografia, senha do admin e do banco e token da API vêm do `.env`, nunca do compose.

A letra L, o contexto. A lição que veio de fora (`docs/analise-aula-vibe-coding.md`): o que o Claude Code entrega depende do que ele lê antes do primeiro comando. Por isso o wizard fecha gerando o prompt de kickoff da primeira sessão (empresa, aprovador, prazo, legado, destino, estratégia, requisitos da v1, como trabalhar), o prompt de prototipação para a ferramenta de telas, o `INDEX_FILES.md` com uma linha por arquivo dizendo o que é e quando abrir, o `.claude/` do projeto com hooks de teste e lint e duas skills (regras do legado; legado para destino), o `.mcp.json` sem chaves, e o Dockerfile e compose por perfil quando L3 pede. Com isso a primeira sessão começa lendo, não perguntando.

PDF vira markdown citável. O PDF é o formato em que quase toda evidência chega, e lê-lo página a página gasta contexto e perde a origem. O `pdf_para_markdown.py` converte uma vez: cada página vira uma seção com `<!-- pagina N -->`, o cabeçalho traz o SHA-256 do PDF, e o resultado entra no RAG. Três regras fazem o resultado valer: **página sempre** (citação sem página é opinião), **página sem texto não vira texto** — PDF escaneado fica marcado `OCR_REQUERIDO` com o número de caracteres encontrados, e o documento diz «nada foi inventado aqui» — e **segredo não passa**, com token substituído e a conta no cabeçalho. A skill `pdf-para-markdown` explica quando usar isto e quando usar o `extrair_pdf.py`, que é o da cadeia formal de evidência do G1, e é honesta sobre o que o markdown não resolve: tabela e coluna dupla saem como texto corrido, e quando a tabela importa se confere contra o PDF.

UI/UX Pro Max, para a tela do destino. Sete skills vendorizadas de `nextlevelbuilder/ui-ux-pro-max-skill` (MIT, versão 2.13.0, commit registrado no `NOTICE.md`): a principal, `ui-ux-pro-max`, traz base local pesquisável — estilos, paletas por produto, pares de fontes, diretrizes de UX, tipos de gráfico e pilhas — e junto vêm `design`, `design-system`, `ui-styling`, `banner-design`, `brand` e `slides`. Elas respondem *antes* de escrever a tela («qual paleta, qual fonte, qual padrão»), enquanto o Impeccable revisa o que já existe. A ordem não muda: a tela modelo do F0 continua mandando, e nenhuma delas autoriza redesenhar o que o usuário conhece sem `DEC-*`.

O risco aqui era medido, não teórico: com vinte skills a listagem podia cortar alguma, como já aconteceu. As sete entraram com descrição encurtada (a maior caiu de 637 para 89 caracteres, as originais guardadas em `skills/descricoes-originais-uiux.json`) e a listagem foi **medida numa sessão nova**: as vinte aparecem. Material de terceiro entra

com licença e atribuição, ou não entra — o teste falha se o `LICENSE`, o `NOTICE.md` ou a linha de origem sumirem.

O instalador resolve o que falta, pedindo aprovação. Antes ele só reclamava e parava; agora, quando Python 3.11+ ou o CLI `claude` faltam, ele **mostra o comando** que instalaria — escolhido pelo gerenciador da máquina (`apt-get`, `dnf`, `yum`, `pacman`, `zypper`, `brew` no Unix; `winget`, `choco` no Windows; `npm` para o Claude Code) — e **pergunta**. Sem o manifesto na pasta, oferece baixar o repositório com `git clone`.

Três regras que o teste trava: nada é instalado sem aprovação explícita; em `--conferir`, num terminal sem TTY, ou com resposta negativa, ele informa e segue (ou para, quando o item é obrigatório); e `--sim` existe para automação, dizendo na saída que foi ele quem respondeu. Provado nos três caminhos: recusa, ausência de terminal e manifesto ausente — este último parando com código 1 sem clonar nada.

Duas asperezas apareceram na minha própria prova e foram corrigidas: em `--conferir` a mensagem dizia «você disse não» quando ninguém tinha dito nada, e o comando de clone aparecia com `$PWD` sem expandir, mostrando ao usuário uma linha que ele não poderia copiar.

Pré-requisitos, medidos. `PRE-REQUISITOS.md` separa o obrigatório (Python 3.11+, o CLI `claude`, ~50 MB de disco) do opcional (`pypdf` ou `pdfminer.six` para ler PDF, `git` para instalar do GitHub, o corpus para a semântica WLanguage) e diz o que se perde sem cada um. **Nenhuma dependência externa de Python é obrigatória:** isso foi medido varrendo os `import` de todos os scripts, e um teste falha se alguém acrescentar uma — ou se importar `pypdf` fora de um `try/except`.

Instalador completo, nas duas plataformas. `instalar.sh` para Linux e macOS, `instalar.ps1` para Windows — que é onde está o público do WINDEV, e era o item 17 da lista do que faltava. Os dois fazem os mesmos cinco passos e param no primeiro problema dizendo qual é: pré-requisitos (Python 3.11+, o CLI `claude`), corpus no lugar (avisando que sem ele o G0 fica `DEGRADED`), validação do pacote antes de instalar qualquer coisa, instalação pelo marketplace local, e licença. `--conferir` mostra o que aconteceria sem mudar nada, e nenhum dos dois toca em algo fora de `~/.claude` e `~/.wx-claude-code`.

A prova real do instalador achou um detalhe no mesmo dia: em `--conferir` ele escrevia `/tmp/wx-validacao.json`, um caminho fixo, e conferir não deve deixar rastro. Agora usa arquivo temporário próprio, apagado ao sair, e um teste falha se sobrar algo em `/tmp`.

Uma ressalva de honestidade: o `instalar.sh` roda na bateria, em modo conferência, a cada rodada de testes. O `instalar.ps1` **não roda aqui** — não há PowerShell neste ambiente — então o teste confere a estrutura dele (blocos balanceados, os cinco passos, os parâmetros) e a prova real fica pendente numa máquina Windows. O que depende do sistema operacional só se prova contra o sistema operacional.

Fontes inventariadas, não listadas à mão. `FONTES.md` conta e mede o pacote na hora — comandos, agentes, skills, scripts, hooks, referências, modelos, testes, exemplo, documentos e instaladores, com arquivos e linhas por grupo — e um teste falha quando alguém acrescenta arquivo e não roda o gerador. Ele foi o primeiro a pegar a si mesmo: os testes que escrevi para o instalador fizeram o inventário envelhecer no mesmo commit.

A licença continua ligada, e a pergunta achou outro defeito. Perguntado se a ativação por serial ainda funciona, fui conferir em vez de responder de memória: `licenca.py verificar` devolveu licença válida, os dois hooks (`hook` no `PreToolUse`, `hook-sessao` no `SessionStart`) continuam chamando o script, e a chave pública RSA-2048 está no pacote. Mas o comando `/wx-claude-code:licenca`, escrito por mim na 3.20.0, mandava rodar `conferir` e `ativar` — dois subcomandos que **nunca existiram**; os certos são `verificar`, `instalar <serial>` e `maquina`. Comando com subcomando inventado só falha na hora do uso, e por isso entrou um teste que lê o `--help` de cada script e confere a primeira palavra depois dele em toda linha de comando dos dezenove `.md`. Dois testes novos: esse, e um que trava a licença ligada nos hooks para ela não sumir numa refatoração.

O registro achou um defeito no primeiro dia. Na sessão de prova, o resumo mostrou duas negativas do hook durante uma conversão de PDF que não escrevia em lugar nenhum. Fui olhar: a guarda dos anexos considerava qualquer `>` na linha como escrita e depois procurava **qualquer** caminho citado no comando — então `script.py --pdf inputs/x.pdf > /tmp/saida` era negado, embora só lesse. Agora o redirecionamento confere o **alvo** dele e o comando destrutivo confere os operandos; ler de `inputs/` e escrever fora passa, escrever ou apagar dentro continua negado, e um teste trava os cinco casos. É o argumento do registro em uma frase: **excesso de bloqueio é invisível até alguém contar**.

Toda operação deixa registro. Cada script do plugin grava uma linha em `.wx-migration/logs/plugin-AAAA-MM-DD.jsonl`: instante, operação, argumentos, código de saída, duração, e o erro quando houve. As negativas dos hooks entram como operação — é assim que se descobre que um agente vinha tentando escrever onde não devia. Três decisões que valem explicar: **sem** `.wx-migration` **por perto não se grava nada**, para o plugin não sujar diretório que não é projeto dele; **falha de registro nunca derruba a**

operação, porque registro que quebra trabalho é pior que registro ausente; e **segredo não entra**, com argumento de nome suspeito virando `<omitido>` e texto com formato de token substituído antes de gravar — log é justamente onde segredo vaza sem ninguém perceber. `/wx-claude-code:log` mostra o resumo por operação ou as últimas em ordem; o zelador apaga o que envelhece.

K8, backup e replicação — e o que se responde, não se pergunta. A letra K fecha com a pergunta que mais se esquece antes de virar um sistema: RPO e RTO declarados, ferramenta e tipo de backup, destino e se ele fica **fora do servidor do banco**, cifragem por nome de variável, retenção, e a data da **última restauração testada de verdade**. Depois a replicação: tipo, se é síncrona, lag aceito, réplicas com papel e região, failover manual ou automático e com qual ferramenta. E o legado: como é hoje, se o HFSQL tem backup, o que não pode parar, e se já perdeu dado — essa última costuma destravar a conversa.

O plano sai em `.wx-migration/ambiente/backup-e-replicacao.md`, escrito para ser lido na hora ruim, e com duas frases que o documento afirma em vez de perguntar: **réplica assíncrona não é backup**, porque copia o `DROP TABLE` junto em segundos, e **backup nunca restaurado é hipótese**. O validador recusa incoerência: RPO de 15 minutos com backup diário e sem WAL contínuo não passa, failover automático sem ferramenta não passa, backup cifrado sem `chave_ref` não passa. Foi essa conferência que pegou o primeiro erro — no exemplo do próprio plugin.

As 60 respostas em markdown, para os agentes. O `respostas_questionario.md` deixou de ser um despejo do JSON e virou o lugar onde um agente procura antes de perguntar. Abre com um aviso dirigido a eles, traz um **índice por id** com as 60 perguntas e o estado de cada uma (respondido: sim ou não), e cada seção carrega o id no título. Resposta vazia aparece como «não», que é o que autoriza perguntar de novo — e só aquele item, com `/wx-claude-code:pergunta <id>`. No fim, o documento aponta para onde mais procurar: o JSON cru, o plano de backup, o catálogo de artefatos e o mapa de arquivos. O RAG indexa o arquivo, então a busca por id acha o trecho com `arquivo#linha`.

Um comando para cada coisa, e um id para cada pergunta. São dezessete comandos: `questionario` (o fluxo inteiro), `pergunta` (uma pergunta pelo id, sem refazer o resto), `comandos` (o índice de tudo), `converter`, `preflight` (só o G0), `artefato`, `estilo-telas`, `golden`, `pmo`, `equipe`, `ambiente`, `help-wl`, `rag`, `exportar`, `zelador`, `licenca` e `laudo-tokens`. Todo recurso do plugin tem um comando que o invoca, e o teste `test_lista_de_perguntas_cobre_o_modelo_e_todo_recurso_tem_comando` trava isso: comando fora do índice quebra a bateria.

As 59 perguntas do questionário têm id — `0.16` é o aprovador, `F0` a tela modelo, `K7` o n8n, `L6` o esqueleto de ERP, `M` os artefatos — e qualquer uma se refaz sozinha com `/wx-claude-code:pergunta <id>`, que grava só aquele ramo do JSON e reaplica. A lista sai do próprio modelo pelo `listar_perguntas.py`, nunca de uma tabela escrita à mão: item novo no questionário aparece na lista sem ninguém lembrar de editar um documento.

Legado E/OU, destino livre — e o que não muda. O plugin existe para converter WLanguage: WINDEV, WEBDEV e WINDEV Mobile para outra linguagem. Isso é pétreo e não sai daqui. O que abriu foi a borda: `projeto.produtos` aceita **um ou mais** de `windev`, `webdev`, `windev-mobile`, `php`, `outra` — E/OU, não E — com `projeto.principal` dizendo qual manda quando duas evidências discordam, e `projeto.legado_outra` descrevendo a linguagem quando não for nenhuma das conhecidas. Do outro lado, o destino não está preso à lista: `H_backend.perfil = "outra"` aceita qualquer linguagem e framework, e o processo de conversão sai genérico, com o G3 detalhando. O validador recusa produto desconhecido, `principal` fora da lista de produtos, e `outra` sem linguagem preenchida — abrir a borda não é abrir mão da conferência.

M, os artefatos. Nem tudo o que importa está no projeto WX. Anotação de reunião, PDF com as classes OOP, `.sql` que o financeiro roda por fora, modelo de relatório impresso, manual, contrato de API, código PHP: cada um é submetido pelo `arquivar_artefato.py`, um por vez, e vai para `artefatos/<tipo>/`. O que faz esse bloco valer é a pergunta **onde usar**, obrigatória: em que gate e em que arquivo do destino aquele artefato entra. Sem ela o script recusa, porque artefato sem destino declarado vira arquivo que ninguém abre. O script também confere segredo (recusa texto com token ou chave privada), calcula o SHA-256, e recusa sobrescrever um arquivo já arquivado com outro conteúdo — reenviar o mesmo arquivo não duplica. `CATALOGO.md` e `registro.json` saem dos fatos e não se editam; a pasta é somente leitura, com hook que recusa escrita, do mesmo jeito que `inputs/`. A diferença entre as duas: `inputs/` é a evidência do WX que o G0 inventaria, `artefatos/` é o que o cliente mandou por fora.

PHP, nos dois sentidos. A skill `php-legado-e-destino` cobre ler um sistema PHP legado (detectar a era e o estilo, grafo de `include`, onde a regra se esconde no PHP procedural, as armadilhas que mudam o resultado — comparação frouxa, dinheiro em `float`, `empty("0")`, encoding) e usar PHP 8.3 como destino (perfil `php`, Laravel 11 ou Symfony 7, com a tabela WLanguage → PHP e as regras do código gerado: dinheiro nunca em `float`, consulta sempre parametrizada, `strict_types`). Legado PHP se declara em `projeto.legado_php` e o código entra como artefato `codigo-php`.

L6, o esqueleto de ERP. Veio de um pacote de oito skills pesquisado no skills.sh pelo dono do projeto (`skills/LEIA-ME-erp.md`): contabilidade, estoque, fiscal brasileiro, multiempresa, alçadas, LGPD, integrações e o WLanguage lido como ERP. Marcado sim, o questionário gera na raiz a árvore que o pacote descreve, preenchida com as respostas: `AGENTS.md` (ordem de leitura, regras que não se negociam, módulo → pasta → skill), `CONTEXT.md` (finalidade, objetivos, recursos e módulos do bloco 0), `CONTEXT-MAP.md`, `UBIQUITOUS_LANGUAGE.md`, `ARCHITECTURE.md`, `SECURITY.md`, quatro ADRs (monólito modular, multiempresa, auditoria e outbox, fiscal), um `docs/domain/<módulo>.md` por módulo, `database/` com migração e rollback pareados, `src/<módulo>/`, `tests/` por camada, `scripts/` e quatro workflows. O `CLAUDE.md` gerado ganha a seção «Skills de ERP» com a tabela módulo → skill, e a sessão carrega a skill certa antes de mexer no módulo (provado no print 33). Multiempresa e fiscal em «não» viram ADRs que dizem que não há, e o que custa entrar depois. Na 3.18.0 o esqueleto ganhou o modelo de ameaças STRIDE por módulo, os contratos OpenAPI e AsyncAPI, ERD e dicionário de dados, invariantes e fluxos consolidados, runbooks de incidente e de backup, e as regras de tabela (dinheiro em `NUMERIC(19,4)`, `TIMESTAMPZ`, constraint no banco, índice por chave estrangeira, `FOR UPDATE` no saldo). E `docs/skills-recomendadas.md`: as skills do catálogo skills.sh que cabem nas respostas, com o comando e a ressalva de ler e fixar versão antes; o plugin não as instala (`references/skills-sh.md` explica por quê).

Sobre a senha. O wizard não pergunta a senha nem o token, e o script recusa o questionário se algum vier preenchido: a regra do projeto é senha nunca em texto puro. Você informa o **nome** da variável de ambiente ou do segredo (`GITHUB_TOKEN`, por exemplo) e configura o valor na máquina que fará o push. Se colar a senha na conversa por engano, revogue-a.

Letras A a L:

LETRA	PERGUNTA
A	caminho do <code>.SQL</code> , dialeto, versão do banco, encoding, collation
B	PDF só dos códigos; é pesquisável?
C	PDF só das interfaces
D	PDF só das queries
E	PDF completo
F	qualidade das telas com o Impeccable: a tela principal como modelo (F0), oito subperguntas de ERP (quem opera, teclado, grids, formulários, formatos, impressão, estados, acessibilidade) e depois paleta, tema, tipografia, preservar ou redesenhar
G	usar o corpus do Help WLanguage 12k? há override da sua versão?
H	para qual linguagem converter o backend (capítulo 7)
I	para qual linguagem e plataforma converter o frontend (capítulo 7)
J	ativar a economia de tokens?
K	ambiente: privilégios (sudo ou root); Rust/Cargo (versão mínima, hoje 1.98 no modelo); PostgreSQL, MySQL, MariaDB e Supabase atualizados, cada um marcável, com login do superusuário e papéis por nível; ligar o projeto ao GitHub (criar, remote, branch, CI); e n8n, sim ou não , com cada item da integração (banco, admin, chave, API do projeto, eventos, webhooks, fluxos)
L	contexto do Claude Code e implantação: requisitos da v1, prototipação (Stitch ou Figma), alvo de deploy com Dockerfile e compose, hooks de teste e lint do projeto, MCP e skills. Gera o kickoff, o <code>INDEX_FILES.md</code> e o <code>.claude/</code> do projeto

E duas perguntas de governança no fim: versão e idioma do WX; modo (`inventário`, `plano`, `piloto`, `completo`). Quem aprova já foi o 0.16.

O que sai:

.wx-migration/
questionario.json suas respostas
respostas_questionario.md todas as respostas legíveis, com o aprovador no topo; o CLAUDE.md aponta para ele
wx-inputs.manifest.json manifesto que o pré-flight lê
conversion.config.json modo, destino, fidelidade
gaps.md, traceability.csv vazios, prontos para o G1
empresa.md softhouse, diretores, endereço, logotipos, finalidade, objetivos, pessoal
processo-de-conversao.md o que cada peça vira e a estratégia (letras H e I)
entrega.json GitHub, branch, usuário, nome da credencial, diretório de destino
ambiente.md, ambiente/n8n/ (letra K) instalador, SQL dos papéis do banco, .env.exemplo e
prompts/ kickoff.md e prototipacao.md (letra L)
pmo/projeto.json prazo final, marcos, orçamento financeiro (o pmo.py iniciar lê)
pmo/cronograma.md, organograma.md, fluxograma.md, riscos.md
CLAUDE.md regras do projeto; aponta para INDEX_FILES.md e para as respostas (estilo de resposta se J = sim)
INDEX_FILES.md mapa de arquivos, regrado sempre
DESIGN.md sistema de design: tela modelo, botões, cores, fundo (se F = sim)
PRODUCT.md quem opera e em que condições (F1)
.claude/ settings.json com hooks, hooks/, skills/regras-do-legado e legado-para-destino
.mcp.json, Dockerfile, docker-compose.yml, .gitignore quando L5 e L3 pedem
AGENTS.md, CONTEXT.md, CONTEXT-MAP.md, UBIQUITOUS_LANGUAGE.md, ARCHITECTURE.md, SECURITY.md, CHANGELOG.md, .editorconfig
docs/{PRD,ROADMAP,BACKLOG}.md, docs/adr/0001-0004, docs/domain/<módulo>.md, docs/{data,api,security,operations,testing}/
database/{schema,migrations,seeds,views,procedures,rollback}, src/<módulo>/, tests/<camada>/, scripts/, .github/workflows/
docs/security/threat-model.md, docs/api/openapi.yaml, events.asyncapi.yaml, docs/data/erd.md, data-dictionary.md, docs/runbooks/
docs/skills-recomendadas.md skills do catálogo skills.sh que cabem nas respostas; o plugin não instala
artefatos/CATALOGO.md o que o cliente mandou por fora, por tipo, com onde usar e hash (bloco M)
artefatos/<tipo>/ o arquivo em si; pasta somente leitura, só arquivar_artefato.py escreve
esqueleto de ERP, quando L6 = sim (62 arquivos no exemplo)

Depois do `pmo.py iniciar` e do fechamento de sprints, `pmo/` ganha ainda `backlog.md`, `base_de_conhecimento.md`, `relatorio.md` e `painel.html`.

A letra F num ERP. Começa pela tela modelo (F0): a captura da tela principal do projeto, o que preservar e o que pode mudar, que vira a seção «Tela modelo» do `DESIGN.md` e a referência do `critique` de toda tela nova. Paleta vem depois. As oito subperguntas (F1 a F8) alimentam o `PRODUCT.md` e seções próprias do `DESIGN.md`, cada uma ligada ao comando do Impeccable que a consome: `shape` para grids, `harden` para formulários e estados, `typeset` para números e moeda, `layout` para impressão, `audit` para acessibilidade. Cinco subperguntas a mais tratam dos botões e do fundo: F9 vocabulário (INCLUIR, ALTERAR, EXCLUIR, GRAVAR, SELECIONAR REGISTRO, VOLTAR, CANCELAR, DUPLICAR, ou a forma em substantivo) e as mensagens exatas; F10 posição das barras em relação à grade e aos campos; F11 ícone por ação; F12 cor por ação, contorno ou preenchido; F13 fundo em cor hexadecimal ou rgb, textura ou imagem. As três viram uma tabela por ação no `DESIGN.md`, que os agentes seguem letra por letra. Uma tela só está pronta quando passa por `polish` e `audit` e atende as seções que a afetam. Detalhe em `references/qualidade-erp.md`.

Repetir o wizard é seguro: o script nunca sobrescreve arquivo que já existe. Para refazer do zero, apague `.wx-migration/` antes.

Sem sessão interativa (`claude -p`), o wizard faz a mesma coisa em texto, uma letra por turno, e você continua com `claude -c "resposta"`.

7. Como definir a linguagem e a plataforma de destino

Esta é a decisão que mais muda o projeto, e por isso o wizard **orienta antes de perguntar**, na letra H.

Se você já sabe, responda a linguagem e siga para framework, banco e implantação em uma frase. A versão do toolchain e do banco a instalar entram na letra K; o alvo de deploy, Dockerfile e compose, na L3.

Se não sabe, o wizard faz quatro perguntas de sinal, uma por vez:

- 1. Quem vai manter o código depois: a equipe WINDEV de hoje ou outra?
- 2. O produto é desktop, web ou mobile?
- 3. Volume e desempenho importam, ou o prazo manda?
- 4. Há linguagem já em uso na empresa?

Com os sinais, mostra **três opções com o porquê em uma frase**, a recomendada primeiro. Estas três estão sempre presentes:

PERFIL	GANHA	CUSTA	SERVE PARA
Rust (Axum + PostgreSQL)	desempenho previsível, binário único, erros pegos em compilação	curva alta, equipe rara	volume alto, motor de cálculo, quem já usa o PhxSql
Python (FastAPI + PostgreSQL)	entrega rápida, biblioteca para fiscal, relatório e dados	desempenho por processo, deploy com runtime	sistemas de gestão que vão evoluir rápido
C# (.NET 8) + WL_C#	a biblioteca WL_C# porta mais de 480 funções do WLanguage com o mesmo nome; tradução das procedures quase mecânica	HFSQL e telas ficam fora da biblioteca; código fechado	a equipe WINDEV que vai manter o código; desktop Windows

Go, Java e Node entram quando os sinais apontarem. A escolha é sua e vira **DEC-0001** na abertura do G3.

Plataforma e frontend (letra I). React (TypeScript) é o padrão para web. Blazor se H foi C#; Flutter se há Android e iOS; Tauri (Rust + React) se o produto continua desktop; Vue ou Svelte para equipes pequenas. Depois: plataformas (web, desktop, Android, iOS), navegadores e dispositivos mínimos.

Tabela de decisão (a linha que mais casa é a recomendação):

SE...	BACKEND	FRONTEND
a equipe WINDEV de hoje mantém e quer a menor mudança	C# + WL_C#	Blazor ou React
é WEBDEV, ou vai para a web, e o time de front vai crescer	Python ou Node	React
há cálculo pesado, volume alto ou o motor é o PhxSql	Rust	React, ou Tauri se desktop
é WINDEV Mobile com Android e iOS	Python ou Go (API)	Flutter
muito relatório, fiscal e integração, e o prazo manda	Python	React
já existe Java ou .NET na empresa	Java ou C#	React ou Blazor

Sobre o WL_C#. É a biblioteca de Bernard Sobra (<https://bernardsobra.github.io/WL-web/>). O plugin traz um índice de 261 funções lido do `WL.dll` 1.0 e o hash da release; o DLL você baixa da release oficial, e o especialista de funções padrão marca cada função como `equivalente`, `adaptar` ou `substituir`. HFSQL, telas, comunicação e relatórios seguem pelos outros especialistas, em qualquer perfil.

Duas regras que não mudam com o perfil. O banco é escolhido em H e instalado em K; o G3 confirma ou muda a decisão (`DEC-*`). E regra de negócio não muda de comportamento por causa da linguagem: o golden master compara o novo com o legado seja qual for o destino.

Como seria a conversão. Junto com as opções, o wizard oferece mostrar o processo de cada uma: o que cada peça do projeto WX vira naquela linguagem, em uma tabela por perfil (procedures, classes, análise HFSQL, queries `.WDR`, janelas, relatórios `.WDE`, funções de string e data), e em que gate isso acontece. Você pode pedir uma opção, todas, ou dizer que já conhece.

Depois da linguagem, ele pergunta a **estratégia**, com a recomendada primeiro:

ESTRATÉGIA	COMO É	QUANDO É RECOMENDADA
tradução assistida	cada procedure vira uma função, na mesma ordem; com a WL_C# é quase mecânica	C# + WL_C#, equipe WINDEV mantendo, prazo curto
reescrita guiada por regras	o inventário extrai as regras BR-* e o código novo nasce delas	Rust ou Python, muito código morto, desenho vai mudar
estrangulamento por módulo	o legado fica no ar e cada módulo migra atrás de uma fachada	sistema grande em produção que não pode parar
ondas com cutover único	ondas no G5 e uma virada só no G7, com paralelo antes	pequeno e médio, banco muda junto

Em I a pergunta se repete para as telas, com o ritmo (tela a tela, módulo a módulo, tudo). O que você confirmou e o que quer diferente vão para `.wx-migration/processo-de-conversao.md`, a primeira versão do que o G3 detalha. Tabelas completas em `references/perfis-de-destino.md`.

8. Licença e serial de ativação

O plugin só executa com um serial válido. Sem ele, a sessão abre com o aviso «sem licença válida», os comandos `/wx-claude-code:*` param na primeira linha e o hook nega a execução dos scripts do plugin e qualquer escrita em `.wx-migration/`. O resto do Claude Code continua normal.

Instalar o serial que você recebeu:

```
python3 "$CLAUDE_PLUGIN_ROOT/skills/conversao-wx/scripts/licenca.py" instalar "WX 2..."  
python3 "$CLAUDE_PLUGIN_ROOT/skills/conversao-wx/scripts/licenca.py" verificar
```

Ele fica em `~/.wx-claude-code/licenca` (ou onde `$WX_LICENCA` apontar). Se o serial for preso a uma máquina, informe ao distribuidor o resultado de `licenca.py maquina` antes de pedir o seu.

Como funciona. O serial é assinado com RSA-2048 pela chave privada de quem distribui; o plugin traz só a chave pública e não consegue emitir nem forjar serial. Um byte alterado invalida a assinatura. Estados possíveis: `valida`, `ausente`, `vencida`, `maquina-diferente`, `assinatura-invalida`, `formato-invalido`, `chave-ausente`.

O que isso protege. O plugin é texto, e quem instala lê tudo. O serial e os hooks são dissuasão para o cliente honesto, não muralha: quem apagar o hook remove a trava. A proteção de verdade é servir o corpus e os agentes de um servidor seu, com o serial conferido a cada chamada. Está explicado, com os comandos de quem distribui, em `licenca/LEIA-ME.md`.

Marca d'água. Com licença válida, o `CLAUDE.md` e o `empresa.md` gerados dizem para quem o plugin foi licenciado.

Apêndice: problemas comuns

- **BLOCKED no G0.** Leia `preflight/runs/<run>/report.md`; cada erro diz o grupo e o arquivo. Enquanto estiver bloqueado, o hook do plugin nega qualquer escrita de código fora de `.wx-migration/`.
- **Skill não aparece na sessão.** Descrição acima de 300 caracteres some da listagem; o validador avisa.
- **Corpus com hash divergente.** Não use; o zip certo tem 26.750.976 bytes.
- **Orçamento estourado.** `rotear_modelo.py` devolve `BLOQUEADO`; decida no PMO com o número.
- **Ciclo PDCA infrutífero não fecha.** Faltou `--proxima`.
- **extrair_pdf.py recusa.** Falta `pypdf` ou `pdfminer.six`; ele diz isso em vez de inventar texto.

O que o plugin não faz

Não lê o formato binário do WX, não faz OCR sozinho, não certifica LGPD, não aprova gate no lugar do humano e não afirma equivalência sem baseline executável do legado.